# 最近公共祖先（LCA）

**节点编号0/1开始都是可以的**

需要维护1.`fa[][]`倍增数组　2.`dep[ ]`深度数组

```
const int N = 5e5 + 10;
int n, m, s, a, b;
vector<int> e[N];  // 邻接表存储图，e[i]存储与节点i相连的节点
int dep[N];        // dep[i]表示节点i的深度
int fa[N][20];     // fa[i][j]表示节点i的第2^j个祖先
//int dis[N];        //每个点到根节点的距离

// 深度优先搜索函数，用于初始化节点的深度和祖先信息
void dfs(int u, int father, int val) {
    dep[u] = dep[father] + 1;  // 设置节点u的深度为其父节点father深度加1
    fa[u][0] = father;         // 节点u的第2^0（即1）个祖先为其父节点father
    //dis[u]=dis[father]+val;  //计算每个点到根节点的距离

    for (int i = 1; i <= 19; i++) {
        // TOP为2^j中j的范围，根据题目范围决定，当然也可以无脑开的稍微大一些
        fa[u][i] = fa[fa[u][i - 1]][i - 1];
        // 节点u的第2^i个祖先，等于节点u的第2^(i-1)个祖先的第2^(i-1)个祖先
    }
    for (int v : e[u]) {    // 遍历与节点u相连的节点v
        if (v != father) {  // 如果v不是u的父节点
            dfs(v, u);      // 递归处理节点v，u作为v的父节点
        }
    }
}

// 计算最近公共祖先（LCA）的函数
int lca(int u, int v) {
    //一、u,v到达同一深度
    if (dep[u] < dep[v]) {  // 如果u的深度小于v的深度，交换u和v，保证u的深度不小于v的深度
        swap(u, v);
    }
    for (int i = 19; i >= 0; i--) {
        if (dep[fa[u][i]] >= dep[v]) {  // 从大到小枚举u的祖先，找到深度{不大于}v的祖先
            u = fa[u][i];  // 将u更新为该祖先       //最终跳到同一层
        }
    }
    if (u == v) {  // 如果u和v相等，说明v就是它们的最近公共祖先
        return v;
    }
    // 二、找到共同祖先
    for (int i = 19; i >= 0; i--) {  // 从大到小枚举u和v的祖先
        if (fa[u][i] != fa[v][i]) {  // 如果u和v的第2^i个祖先{不相等}
            u = fa[u][i];  // u更新为第2^i个祖先
            v = fa[v][i];  // v更新为第2^i个祖先
        }
    }
```

```
    return fa[u][0];  // 返回u的父节点，即为u和v的最近公共祖先
}
ll Dis(ll a,ll b){
    return dis[a]+dis[b]-2*dis[lca(a,b)];
}
```

有权树：

```
//核心代码
vector<edge>g[N];
int dep[N],dis[N],fa[N][20];//dis[j]为 j到根节点的距离
void dfs(int p,int father,int val){
    fa[p][0]=father;
    dep[p]=dep[father]+1;
    val[p]=val[father]+val;
    for(int i=1;i<20;i++){
        fa[p][i]=fa[fa[p][i-1]][i-1];
    }
    for(auto v:g[p]){
        if(v!=father)dfs(v.to,p,v.val);
    }
}
ll Dis(int a,int b){//a,b的最短路长度
    return dis[a]+dis[b]-2*dis[lca(a,b)];
}
```

# 线性基

线性基是**用于处理异或问题**的一种数据结构，核心就一句话：

用**最少的一组数**，能异或出原集合能异或出的**所有结果。**

1. **只针对异或**：不是加法、乘法，是**按位异或**
2. 性质
   - 线性基里的数，**互相异或不会得到 0**
   - 原集合任意数，都能被线性基里的数**异或出来**
   - 线性基**大小最小**，没有冗余
   - 异或和个数 $2^n$ -1
3. 作用
   - 求一组数的**最大异或和**
   - 求能异或出多少种不同值
   - **判断某个数能不能被异或出来**

```
const int MAX_BIT = 63;  // 处理long long范围(0~63位)，int的话用31即可
struct LinearBasis {
    ll d[MAX_BIT + 1];  // d[i]表示第i位为1的线性基元素
    bool flag;          // 标记原集合是否能异或出0（存在重复/线性相关）

    // 初始化线性基
    LinearBasis() {
        fill(d, d + MAX_BIT + 1, 0);
```

```
        flag = false;
    }

    // 插入一个数到线性基
    void insert(ll x) {
        // 从高位到低位遍历
        for (int i = MAX_BIT; i >= 0; --i) {
            if (x & (1LL << i)) {  // 找到x的最高非零位
                if (!d[i]) {       // 该位线性基为空，直接插入
                    d[i] = x;
                    break;
                } else {           // 该位已有基，异或消去该位
                    x ^= d[i];
                }
            }
        }
        // 如果x最终变为0，说明原集合存在线性相关，能异或出0
        if (x == 0) flag = true;
    }

    // 求最大异或和
    ll get_max() {
        ll res = 0;
        // 从高位到低位贪心，尽可能让每一位为1
        for (int i = MAX_BIT; i >= 0; --i) {
            if ((res ^ d[i]) > res) {
                res ^= d[i];
            }
        }
        return res;
    }

    // 求最小异或和
    ll get_min() {
        // 如果能异或出0，最小就是0
        if (flag) return 0;
        // 否则找线性基中最小的元素（最低位的非零基）
        for (int i = 0; i <= MAX_BIT; ++i) {
            if (d[i]) return d[i];
        }
        return 0;  // 空集情况
    }

    // 判断数x是否能被线性基异或出来
    bool check(ll x) {
        if (x == 0) return flag;  // 0需要看是否能异或出
        for (int i = MAX_BIT; i >= 0; --i) {
            if (x & (1LL << i)) {
                x ^= d[i];
                if (x == 0) return true;
            }
        }
        return false;
    }
};

```

```cpp
// 测试样例
int main() {
    LinearBasis lb;
    // 插入一组数
    ll a[] = {1, 2, 4, 8};
    int n = sizeof(a) / sizeof(ll);
    for (int i = 0; i < n; ++i) {
        lb.insert(a[i]);
    }

    cout << "最大异或和: " << lb.get_max() << endl;  // 输出15 (1^2^4^8)
    cout << "最小异或和: " << lb.get_min() << endl;  // 输出1
    cout << "能否异或出3: " << (lb.check(3) ? "是" : "否") << endl;  // 是（1^2）
    cout << "能否异或出5: " << (lb.check(5) ? "是" : "否") << endl;  // 是（1^4）
    cout << "能否异或出7: " << (lb.check(7) ? "是" : "否") << endl;  // 是（1^2^4）
    cout << "能否异或出0: " << (lb.check(0) ? "是" : "否") << endl;  // 否

    return 0;
}
```

# 整除分块

**整除分块**

$$求 \quad \sum_{i=1}^{n}\left\lfloor\frac{n}{i}\right\rfloor$$

例：$n=21$ 时，$\left\lfloor\frac{n}{i}\right\rfloor$ 的值分布在 8 个块内

| $i$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | $\cdots$ | 21 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\left\lfloor\frac{n}{i}\right\rfloor$ | 21 | 10 | 7 | 5 | 4 | 3 | | 2 | | | 1 | | |

**性质1：分块的块数 $\le 2\lfloor\sqrt{n}\rfloor$**

当 $i \le \lfloor\sqrt{n}\rfloor$ 时，$\left\lfloor\frac{n}{i}\right\rfloor$ 有 $\lfloor\sqrt{n}\rfloor$ 种取值。

当 $i > \lfloor\sqrt{n}\rfloor$ 时，$\left\lfloor\frac{n}{i}\right\rfloor \le \lfloor\sqrt{n}\rfloor$，$\left\lfloor\frac{n}{i}\right\rfloor$ 至多有 $\lfloor\sqrt{n}\rfloor$ 种取值。

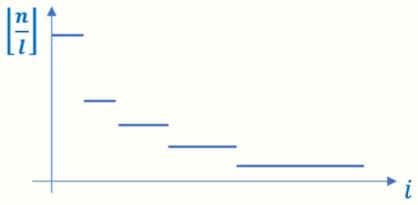
**性质2：$i$ 所在块的右端点为 $\left\lfloor\frac{n}{\left\lfloor\frac{n}{i}\right\rfloor}\right\rfloor$**

$i$ 所在块的值 $k=\left\lfloor\frac{n}{i}\right\rfloor$，则 $k \le \frac{n}{i}$，则 $\left\lfloor\frac{n}{k}\right\rfloor \ge \left\lfloor\frac{n}{\frac{n}{i}}\right\rfloor = \lfloor i\rfloor = i$

所以 $i_{max} = \left\lfloor\frac{n}{k}\right\rfloor = \left\lfloor\frac{n}{\left\lfloor\frac{n}{i}\right\rfloor}\right\rfloor$。代码实现时，右端点 $r=n/(n/l)$;

$$求 \quad \sum_{i=1}^{n} f(i)\left\lfloor\frac{n}{i}\right\rfloor$$

先预处理出$f(i)$的前缀和 $s(i)=\sum_{j=1}^{i} f(j)$，
再枚举每一块 $[l,r]$，累加每块的贡献。




$$
\begin{aligned}
&for(int\ l=1;\ l\le n;\ l=r+1)\{\\
&\quad \boxed{r=n/(n/l);}\\
&\quad res\ +=\ \underbrace{(s(r)-s(l-1))}_{f(i)} * \left(\frac{n}{l}\right);\\
&\}
\end{aligned}
$$

时间复杂度：$O(\sqrt{n})$

**根据上图我们枚举i过程只会枚举1，2，3，4，5，6，8，11。**

```
//整除分块
for(int l=1,r;l<=n;l=r+1){//k=[l~r]  floor(n/k)一样
    r=n/(n/l);
    //执行一些操作
}

```

数学上，`ceil(b/k)`（`b` 除以 `k` 的上取整）可以等价转化为下取整形式：

$\lceil\frac{b}{k}\rceil = \frac{b+k-1}{k} = \lfloor\frac{b-1}{k}\rfloor + 1$

```
for(int l=1,r;l<=n;l=r+1){//k=[l~r]  ceil(n/k)一样
    r=(n-1)/((n-1)/l);
    int r2=r+1;//上取整一样 上限
}
```

# 高精度

## X

```
const int N=1e6;
typedef long long ll;

ll a[N],b[N],c[N];//第一个数，第二个数，结果
ll la,lb,lc;

void gcf(){
    for(ll i=0;i<la;i++){//从低位到高位*
        for(ll j=0;j<lb;j++){
            c[i+j]+=a[i]*b[j];//存数
            c[i+j+1]+=c[i+j]/10;//进位
            c[i+j]%=10;//取余
        }
    }
    while(lc && c[lc]==0) lc--;//清空前导0
}

int main()
{
    string s1,s2;
    cin>>s1>>s2;
    la=s1.size();
    lb=s2.size();
    lc=la+lb;
    reverse(s1.begin(),s1.end());
    reverse(s2.begin(),s2.end());

    for(ll i=0;i<la;i++) a[i]=s1[i]-'0';
    for(ll i=0;i<lb;i++) b[i]=s2[i]-'0';

    gcf();
    for(ll i=lc;i>=0;i--)cout<<c[i];//输出结果
    return 0;
}
```

## /

```
string div(string a, ll b, ll& rem) {
    vector<int> res;
    rem = 0;
    for (char c : a) {
        rem = rem * 10 + (c - '0');
```

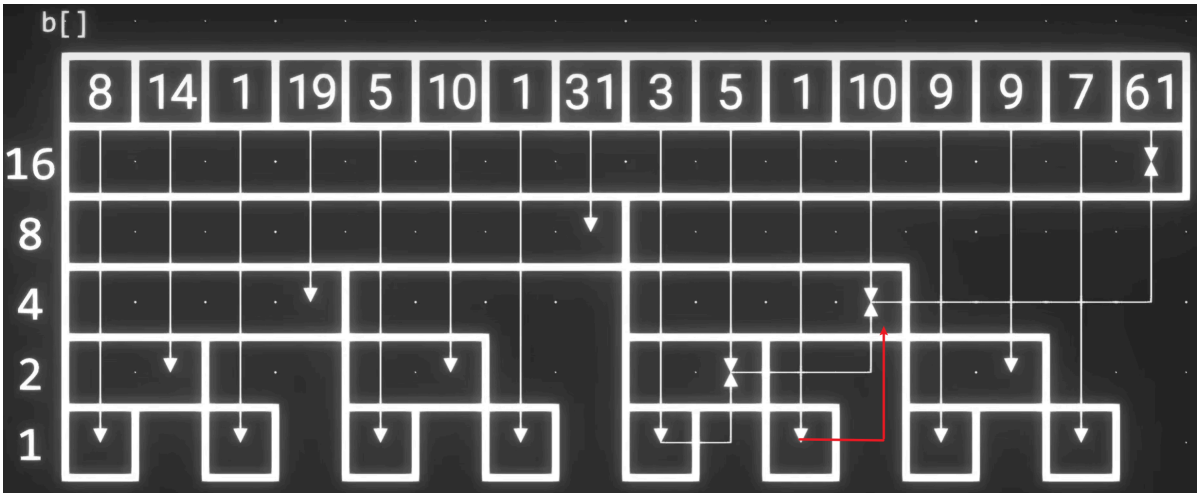
```
        res.push_back(rem / b);
        rem %= b;
    }
    int p = 0;// 去前导零
    while (p < res.size() && res[p] == 0) p++;
    if (p == res.size()) return "0";
    string ans;// 拼接结果
    for (; p < res.size(); p++) ans += (char)(res[p] + '0');
    return ans;
}

int main()
{
    string a; ll b, rem;
    cin >> a >> b;
    string q = div(a, b, rem);
    cout << q << '\n' << rem << endl;

    return 0;
}
```

# 树状数组

**（下标从1开始）**




高效处理**区间查询**和**单点更新**的数据结构。以下是关于树状数组的详细介绍：

```
int n,m,s[500005];
int lowbit(int x){//返回二进制最低非零位    lowbit(23) 11000    2的4次方
    return x&-x;
}
//原理:    1.lowbit(x)==s[x](以x结尾)的区间长度
void change(int x,int k){    //  *向后修
    while(x<=n)s[x]+=k,x+=lowbit(x);
}
int query(int x){// *向前查
    int t=0;
    while(x)t+=s[x],x-=lowbit(x);
    return t;
}
int getsum(int l,int r){//区间查询
    return query(r)-query(l-1);
```

```
}
void solve(){
    cin>>n>>m;
    int op,x,y;
    for(int i=1;i<=n;i++){
        cin>>y;change(i,y);
    }
    for(int i=1;i<=m;i++){
        cin>>op>>x>>y;
        if(op==1)change(x,y);
        else cout<<query(y)-query(x-1);//区间查询
    }
}
```

区间修改，单点查询： 树状数组维护**差分数组**

```
int n,m,tree[500005];
void solve(){
    cin>>n>>m;
    int op,x,y;
    for(int i=1;i<=n;i++){
        cin>>i>>x;
        change(i,x);
        change(i+1,-x);//差分
    }
    for(int i=1;i<=m;i++){
        cin>>op;
        if(op==1){//区间修改
            cin>>l>>r>>x;
            change(l,x);
            change(r+1,-x);
        }
        else{
            cin>>x;
            cout<<query(x);//单点查询
        }
    }
}
```

## 动态开点

**开N课树状数组** N=2e5 大数

## 树状数组上倍增

原理:

树状数组**节点块状的** 预处理出区间和。**倍增思想就用于计算前缀和**

```
int now = 0, cnt = 0;
int b;
for (b = 1; b <= n; b <<= 1);
for (b >>= 1; b; b >>= 1) {//b=2的幂
```

```
    int nxt = now + b;//net=新前缀和长度
    int tmp = 0;
    //  关键：只看 nxt 这一个树状数组节点的哈希表，不重新算整个前缀
    for (int c : vec) {
        auto it = colTree[nxt].find(c);
        if (it != colTree[nxt].end()) tmp += it->second;
    }
    // 判断 [lim+1, nxt] 是否合法
    if (nxt < lim || (cnt + tmp) - base == nxt - lim) {
        now = nxt;
        cnt += tmp; //  累加当前节点的信息
    }
}
```

## 树状数组上二分

**整体二分的应用场**

1. 单个问题的答案具有单调性，当答案固定时，问题的要求可能得到满足或者无法满足
2. 如果要求得到满足，那么答案变差，要求依然能满足，此时去寻找更好的答案
3. 如果要求无法满足，那么答案变好，要求也无法满足，此时去寻找更差的答案
4. 问题数量较多，无法对每个问题都执行一次二分答案，但是所有查询允许**离线处理**

**整体二分的基本思想**

1. 让修改操作适应当前的答案范围，调整好数据状况，然后查看每个问题的要求是否达标
2. **多个问题共享这一轮的修改操作后的数据状况，避免重复劳动**
3. 达标的问题去往左组，不达标的问题去往右组，然后左右分别递归，直到解决所有的问题

**整体二分的时间复杂度**

假设，答案范围为`n`，问题数量为`q`，单次修改或查询为`log n`

那么，总的时间复杂度为O((n + q) * log n * log n)

O((n + q) * log n *数据结构的单次修改或查询复杂度)

**求区间第k小**

```
int n,m;
PII arr[N];//位置i,数值v
int qid[N],l[N],r[N],k[N];//查询列表
int tree[N]; //树状数组
int lset[N],rset[N];//整体二分
int ans[N];
int lowbit (int i){return i&-i;};
void add(int i,int v){
    for(i;i<=n;i+=lowbit(i))tree[i]+=v;
}
int sum(int i){
    int res=0;
    for(;i;i-=lowbit(i)){
```

```
        res+=tree[i];
    }
    return res;
}
int query(int l,int r){
    return sum(r)-sum(l-1);
}
void compute(int ql,int qr,int vl,int vr){
    if(ql>qr)return;
    if(vl==vr){
        for(int i=ql;i<=qr;i++){
            ans[qid[i]]=arr[vl].second;
        }
    }else{
        //标记
        int mid=(vl+vr)/2;
        for(int i=vl;i<=mid;i++){
            add(arr[i].first,1);
        }
        // 检查每个问题并划分左右
        int lsiz=0,rsiz=0;
        for(int i=ql;i<=qr;i++){
            int id=qid[i];
            int satisfy=query(l[id],r[id]);
            if(satisfy>=k[id]){
                lset[++lsiz]=id;//b,c,e
            }else{
                k[id]-=satisfy;
                rset[++rsiz]=id;//a,b
            }
        }
        for(int i=1;i<=lsiz;i++){//[a,b,c,d,e]->[a,c,e,a,b]
            qid[ql+i-1]=lset[i];//划分左边问题
        }
        for(int i=1;i<=rsiz;i++){//划分右边问题
            qid[ql+lsiz+i-1]=rset[i];
        }
        //回撤标记
        for(int i=vl;i<=mid;i++){
            add(arr[i].first,-1);
        }
        //两边各自递归
        compute(ql,ql+lsiz-1,vl,mid);
        compute(ql+lsiz,qr,mid+1,vr);
    }

}
void solve(){
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        arr[i].first=i;
        cin>>arr[i].second;
    }
    for(int i=1;i<=n;i++){
        qid[i]=i;
        cin>>l[i]>>r[i]>>k[i];
```

```
    }
    sort(arr+1,arr+1+n,[](auto a,auto b){
        return a.second<b.second;
    });
    compute(1,m,1,n);
    for(int i=1;i<=n;i++)cout<<ans[i]<<endl;;
}
```

**写法二**

```
int n,m;
PII arr[N];//位置i,数值v
int qid[N],l[N],r[N],k[N];//查询列表
int tree[N]; //树状数组
int lset[N],rset[N],used=0;//整体二分
int ans[N];
int lowbit (int i){return i&-i;};
void add(int i,int v){
    for(i;i<=n;i+=lowbit(i))tree[i]+=v;
}
int sum(int i){
    int res=0;
    for(;i;i-=lowbit(i)){
        res+=tree[i];
    }
    return res;
}
int query(int l,int r){
    return sum(r)-sum(l-1);
}
void compute(int ql,int qr,int vl,int vr){
    if(ql>qr)return;
    if(vl==vr){
        for(int i=ql;i<=qr;i++){
            ans[qid[i]]=arr[vl].second;
        }
    }else{
        int mid=(vl+vr)/2;
        //标记
        while (used < mid) {
            used++;
            add(arr[used].first, 1);
        }
        while (used > mid) {
            add(arr[used].first, -1);
            used--;
        }
        //划分
        int lsiz=0,rsiz=0;
        for(int i=ql;i<=qr;i++){
            int id=qid[i];
            int satisfy=query(l[id],r[id]);
            if(satisfy>=k[id]){
                lset[++lsiz]=id;//b,c,e
            }else{
```

```
                k[id]-=satisfy;
                rset[++rsiz]=id;//a,b
            }
        }
        for(int i=1;i<=lsiz;i++){//[a,b,c,d,e]->[a,c,e,a,b]
            qid[ql+i-1]=lset[i];
        }
        for(int i=1;i<=rsiz;i++){
            qid[ql+lsiz+i-1]=rset[i];
        }
        //两边各自递归
        compute(ql,ql+lsiz-1,vl,mid);
        compute(ql+lsiz,qr,mid+1,vr);
    }
}
void solve(){
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        arr[i].first=i;
        cin>>arr[i].second;
    }
    for(int i=1;i<=n;i++){
        qid[i]=i;
        cin>>l[i]>>r[i]>>k[i];
    }
    sort(arr+1,arr+1+n,[](auto a,auto b){
        return a.second<b.second;
    });
    compute(1,m,1,n);
    for(int i=1;i<=n;i++)cout<<ans[i]<<endl;;
}
```

# 可持续化线段树

## 一、维护下标历史版本改、查

如题，你需要维护这样的一个长度为 $N$ 的数组，支持如下几种操作

1. 在某个历史版本上修改某一个位置上的值

2. 访问某个历史版本上的某一位置的值

```
int n,m,a[N];
int root[N],tot; //根节点,节点个数
int ls[N*25],rs[N*25],val[N*25];
void build(int &u,int l,int r){ //建树
    u=++tot; //动态开点
    if(l==r){val[u]=a[l]; return;}
    int mid=(l+r)>>1;
    build(ls[u],l,mid);
    build(rs[u],mid+1,r);
}

void change(int &u,int v,int l,int r,int p,int x){//点修
```

```
    u=++tot; //动态开点
    ls[u]=ls[v]; rs[u]=rs[v]; val[u]=val[v];
    if(l==r){val[u]=x; return;}
    int mid=(l+r)>>1;
    if(p<=mid) change(ls[u],ls[v],l,mid,p,x);
    else change(rs[u],rs[v],mid+1,r,p,x);
}

int query(int u,int l,int r,int p){ //点查
    if(l==r) return val[u];
    int mid=(l+r)>>1;
    if(p<=mid) return query(ls[u],l,mid,p);
    else return query(rs[u],mid+1,r,p);
}

int main(){
    scanf("%d%d", &n,&m);
    for(int i=1;i<=n;i++) scanf("%d",a+i);
    build(root[0],1,n);
    for(int i=1;i<=m;i++){
        int ver,op,p,x; scanf("%d%d", &ver,&op);
        if(op==1){
            scanf("%d%d", &p,&x);
            change(root[i],root[ver],1,n,p,x);
        }
        else{
            scanf("%d", &p);
            root[i]=root[ver];
            printf("%d\n",query(root[ver],1,n,p));
        }
    }
}
```

## 二、维护值域

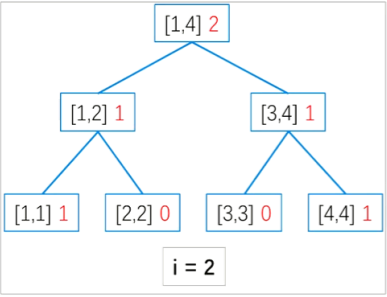
**3. query()**
查询：在主席树上查询区间 $[l,r]$ 第 k 小

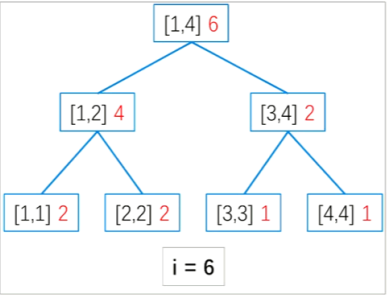
简化：求 $[1,r]$ 区间内的第 k 小 。找到插入 r 时的历史版本，在权值线段树上二分搜索就行了。
扩展：求 $[l,r]$ 区间内的第 k 小 。我们想到了**前缀和**，$[l,r]$ 的信息 $=[1,r]$ 的信息 $-[1,l-1]$ 的信息。

参数：x为前一版本的节点指针，y为当前版本的节点指针。

```
int query(int x,int y,int l,int r,int k){
  if(l==r) return l;
  int m=l+r>>1;          // 双指针同步搜索
  int s=tr[lc(y)].s-tr[lc(x)].s;
  if(k<=s) return query(lc(x),lc(y),l,m,k);
  else return query(rc(x),rc(y),m+1,r,k-s);
}
```

```
scanf("%d%d%d",&l,&r,&k);
query(root[l-1],root[r],1,n,k);
```




a[ ]: 1 4 2 2 3 1
查询: 3 6 4

query (root[2], root[6], k=4)
    s=4-1=3;
    (k>s) query (rc(x), rc(y), k-s)
query ([3,4], [3,4], k=1)
    s=1-0=1;
    (k>=s) query (lc(x), lc(y), k=1)
query ([3,3], [3,3], k=1)
    (l==m) return 3

**数组**

```
#define N 200005
int n,m,a[N],b[N];
int root[N],tot; //根节点,节点个数
int ls[N*20],rs[N*20],sum[N*20];//sum:区间数的出现次数之和
void build(int &u,int l,int r){ //建空树
    u=++tot; //动态开点
    if(l==r)return;
    int mid=(l+r)>>1;
    build(ls[u],l,mid);
    build(rs[u],mid+1,r);
}
void change(int &u,int v,int l,int r,int p){ //u新版本根,v前版本根
    u=++tot; //动态开点
    ls[u]=ls[v]; rs[u]=rs[v]; sum[u]=sum[v]+1;//构造新节点
    if(l==r) return;  //双指针同步搜索
    int mid=(l+r)>>1;
    if(p<=mid) change(ls[u],ls[v],l,mid,p);
    else change(rs[u],rs[v],mid+1,r,p);
}
int query(int u,int v,int l,int r,int k){ //点查
    if(l==r)return l;  //双指针同步搜索
    int s=sum[ls[u]]-sum[ls[v]];
    int mid=(l+r)>>1;
    if(s>=k)return query(ls[u],ls[v],l,mid,k);
    else return query(rs[u],rs[v],mid+1,r,k-s);
}
// 查询区间[l,r]中值在[L, R]范围内的点数
int query_range(int u, int v, int l, int r, int L, int R) {
    if (!v|| L > R) return 0;
    if (L <= l && r <= R) return ls[v] - rs[u];
    int mid = (l + r) >> 1, res = 0;
    if (L <= mid) res += query_range(ls[u], ls[v], l, mid, L, R);
    if (R > mid) res += query_range(rs[u], rs[v], mid + 1, r, L, R);
    return res;
}

```

**结构体**

```
#define lc(x) tr[x].ch[0]
#define rc(x) tr[x].ch[1]
const int N=1e5+5;
int n,m,a[N];
vector<int>v;
struct node{
    int ch[2] , s;
}tr[N*22];
int root[N],idx;//根节点
void build(int &x,int l,int r){//空树
    x=++idx;//编号
    if(l==r)return;
    int m=l+r>>1;
    build(lc(x),l,m);
```

```
    build(rc(x),m+1,m);
}
void insert(int x,int &y,int l,int r,int v){//x前版本根，y新版本根
    y=++idx;tr[y]=tr[x];tr[y].s++;//编号，新节点赋值原节点
    if(l==r)return;
    int m=l+r>>1;
    if(v<=m)insert(lc(x),lc(y),l,m,v);
    else insert(rc(x),rc(y),m+1,r,v);
}
int query(int x,int y,int l,int r,int k){////查询[l~r]第k小的数
    if(l==r)return 1;
    int m=l+r>>1;
    int s=tr[lc(y)].s-tr[lc(x)].s;//前缀和，相同范围区间
    if(k<=s)return query(lc(x),lc(y),l,m,k);
    else return query(rc(x),rc(y),m+1,r,k-s);
}
// 查询区间[l,r]中值在[L, R]范围内的点数
int query_range(int x, int y, int l, int r, int L, int R) {
    if (!y || L > R) return 0;
    if (L <= l && r <= R) return tr[y].s - tr[x].s;
    int mid = (l + r) >> 1, res = 0;
    if (L <= mid) res += query_range(lc(x), lc(y), l, mid, L, R);
    if (R > mid) res += query_range(rc(x), rc(y), mid + 1, r, L, R);
    return res;
}
void solve(){
    build(root[0],1,n);
    for(int i=1;i<=n;i++)insert(root[i-1],root[i],1,n,a[i]);
    cin>>l>>r;
    query(root[l-1],root[r],l,r,k);//查询[l~r]第k小的数
}
```

# 线段树上二分

美味度排序，**整体二分美味度**，用线段树维护（单价线段树 查询最小花费）是否满足条件

```
const int N=1e5+5,mod=998244353,INF=1e18;
int n,m;
struct node{
    int d,p,l;
};
node juice[N];
int gid[N],money[N],least[N],ans[N];
int used=0,maxp=0;
int cost[N<<2];
int suml[N<<2];
int lset[N],rset[N];
void up(int root){
    suml[root]=suml[root<<1]+suml[root<<1|1];
    cost[root]=cost[root<<1]+cost[root<<1|1];
}
void add(int root, int l, int r, int p, int val){
    if(l == r){
        suml[root] += val;
        cost[root] = l * suml[root];
```

```
        return;
    }
    int mid = (l + r) / 2;
    if(p <= mid) add(root<<1, l, mid, p, val);
    else add(root<<1|1, mid+1, r, p, val);
    up(root);
}

long long query(int root, int l, int r, long long need) {
    if (l == r) {
        return need * l;
    }
    int mid = (l + r) / 2;
    if (suml[root << 1] >= need) {
        return query(root << 1, l, mid, need);
    } else {
        return cost[root << 1] + query(root << 1 | 1, mid + 1, r, need - suml[root << 1]);
    }
}
void compute(int ql,int qr,int vl,int vr){
    if(ql>qr)return;
    if(vl==vr){
        for(int i=ql;i<=qr;i++){
            ans[gid[i]]=vl;
        }
    }else{
        int mid=(vl+vr)/2;
        while(used<mid){
            used++;
            add(1,1,maxp,juice[used].p,juice[used].l);
        }
        while(used>mid){
            add(1,1,maxp,juice[used].p,-juice[used].l);
            used--;
        }
        int lsiz=0,rsiz=0;
        for(int i=ql;i<=qr;i++){
            int id=gid[i];
            if(suml[1]>=least[id]&&query(1,1,maxp,least[id])<=money[id]){
                lset[++lsiz]=id;
            }else{
                rset[++rsiz]=id;
            }
        }
        for(int i=1;i<=lsiz;i++){
            gid[ql+i-1]=lset[i];
        }
        //  唯一修改：这里把 rsiz 改成 lsiz!
        for(int i=1;i<=rsiz;i++){
            gid[ql + lsiz + i - 1] = rset[i];
        }
        compute(ql,ql+lsiz-1,vl,mid);
        compute(ql+lsiz,qr,mid+1,vr);
    }
}
```

```
void solve(){
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        int d,p,l;cin>>d>>p>>l;
        juice[i]={d,p,l};
        maxp=max(maxp,p);
    }
    sort(juice+1,juice+n+1,[&](node a,node b){
        return a.d>b.d;
    });
    for(int i=1;i<=m;i++){
        gid[i]=i;
        cin>>money[i]>>least[i];
    }
    compute(1,m,1,n+1);
    for(int i=1;i<=m;i++){
        if(ans[i]==n+1){
            cout<<-1<<'\n';
        }else{
            cout<<juice[ans[i]].d<<'\n';
        }
    }
}
```